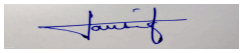
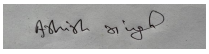
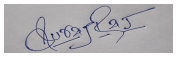
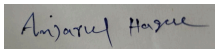
PROJECT DOCUMENTATION

Road Safety & Traffic Incident Analytics Dashboard

Data Analysis • Business Intelligence • Dashboard Development

Submitted By (Group B)

- **Md Tausif Raza —** 
- **Ashish Singh —** 
- **Suraj Raj —** 
- **Sneha Mondal —** 
- **Anjarul Haque —** 

Project Mentorship

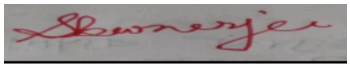
- **Mentor Name:** Subhashish Banerjee
- **Mentor Digital Signature:** 

Table of Contents

1. Executive Summary
2. Introduction & Problem Statement
3. Project Objectives
4. Business Requirements
5. Technology Stack
6. System Architecture & Folder Structure
7. Dataset Description & Data Dictionary
8. Data Cleaning & Preprocessing

9. Exploratory Data Analysis & Python KPIs
10. Dashboard UI & Output Screenshots
11. Findings & Business Insights
12. Recommendations
13. Conclusion & Future Scope
14. GitHub Repository & References
15. Appendix: Full Source Code

---

1. Executive Summary

This project focuses on analyzing road traffic incident data to understand collision severity, emergency response efficiency, and geographic risk patterns. The dataset consists of over 12,000 synthetically generated traffic incident records and emergency response logs. Python and Pandas are utilized for backend data cleaning, transformation, and Exploratory Data Analysis (EDA). The project completely avoids traditional drag-and-drop BI tools in favor of a pure-Python ecosystem, leveraging Matplotlib and Seaborn for advanced data visualization and Streamlit for developing an interactive, real-time web dashboard.
A key feature of this application is its Dataset Upload Functionality, allowing users to dynamically upload incident and response CSV files directly through the UI, making the system highly adaptable to future data without requiring code modifications.

---

2. Introduction & Problem Statement

2.1 Introduction

Road traffic accidents vary by location, time, severity, and vehicle type. Traditional static reporting fails to capture the dynamic nature of these incidents, leading to delays in identifying critical black spots and evaluating emergency response Key Performance Indicators (KPIs). This project converts raw, distributed traffic data into a centralized, decision-support web application.

2.2 Problem Statement

Traffic management authorities and civic planners struggle to track accident hotspots and monitor emergency medical service (EMS) dispatch delays in real-time. The organization needs to understand:

- Where are the highest-risk localities (hotspots)?
- What is the average emergency response time across different zones?
- What are the primary accident types and their severity levels?

- How do incident volumes fluctuate on an hourly basis?

The objective is to transform raw CSV data into a fully interactive, Python-based dashboard capable of dynamic file ingestion and live data entry.

---

3. Project Objectives

16. Build a clean, integrated traffic-incident dataset comprising 12,000+ records.
17. Develop a seamless Dataset Upload Functionality for dynamic data ingestion via the frontend.
18. Identify accident hotspots by geographic locality and temporal distribution.
19. Calculate and visualize Emergency Response Time (Arrival Time - Alert Time).
20. Implement a live Data Entry Form for reporting new accidents in real-time.
21. Create an interactive, dark-themed Streamlit web dashboard.

4. Business Requirements

22.

Business Area	Business Question
Incident Volume	What is the total number of accidents recorded in the dataset?
Casualties	What are the total fatalities and injuries?
Response Time	What is the average emergency response time in minutes?
Hotspots	Which localities experience the highest volume of accidents?
Time Pattern	At what hour of the day do most incidents occur?
Severity	What proportion of accidents are classified as Fatal vs. Minor?

5. Technology Stack

23.

Technology	Purpose
Python 3.x	Core programming language for data logic and execution.
Pandas & NumPy	High-performance data manipulation, joining datasets, and KPI calculations.
Streamlit	Frontend web framework for creating the interactive dashboard and file upload UI.
Matplotlib & Seaborn	Generating advanced statistical graphics and customized dark-themed charts.
Git / GitHub	Source code version control and project deployment repository.

6. System Architecture & Folder Structure

6.1 Model-View-Controller (MVC) Pattern

The project architecture mimics an MVC pattern tailored for data science:

- **Model:** Pandas DataFrames handling the data logic, merging, caching, and state management via `st.session_state`.
- **View:** The Streamlit frontend rendering the metrics, file uploaders, and Matplotlib charts.
- **Controller:** The Python script orchestrating the flow from user file-upload to data processing and UI rendering.

6.2 Project Folder Structure

To ensure clean code management, the project is structured as follows:

```
None
Road_Safety_Analytics/
│
├── data/
│   └── raw/
│       ├── accidents.csv
│       └── responses.csv
│
├── src/
```

```
|   ├── app.py                  # Main Streamlit Dashboard Application
|   └── generate_new_data.py    # Python script to generate 12,000+ dataset
records
|
├── .gitignore                  # Ignores system files like __pycache__
├── README.md                   # Project documentation and run instructions
└── requirements.txt            # Python dependencies (streamlit, pandas,
matplotlib, seaborn)
```

7. Dataset Description & Data Dictionary

Scale: 12,000+ Records
Primary Files: `accidents.csv` and `responses.csv`

7.1 Data Dictionary: accidents.csv

Column Name	Data Type	Description
`accident_id`	String	Unique identifier for each accident (e.g., ACC00001).
`date`	Date	Date of the incident occurrence.
`time`	Time	Time of the incident occurrence.
`locality`	String	Geographic zone (e.g., Kalyani, Kolkata, Howrah).
`accident_type`	String	Type of crash (e.g., Two-wheeler crash, Pedestrian collision).
`severity`	String	Classification (Minor, Serious, Critical, Fatal).
`fatalities`	Integer	Number of deaths.
`injuries`	Integer	Number of non-fatal injuries.

7.2 Data Dictionary: responses.csv

Column Name	Data Type	Description
`response_id`	String	Unique identifier for the emergency response.
`accident_id`	String	Foreign key mapping to accidents.csv.
`alert_time`	Datetime	Timestamp when emergency services were notified.
`arrival_time`	Datetime	Timestamp when emergency services arrived at the scene.

8. Data Cleaning & Preprocessing

The application handles preprocessing dynamically upon file upload. Key operations include:

24. **Datetime Conversion:** Converting string-based `alert_time` and `arrival_time` into pandas `datetime64[ns]` objects.
25. **KPI Calculation:** Deriving `response_time_mins` by subtracting `alert_time` from `arrival_time` and converting total seconds to minutes.
26. **Data Merging:** Performing an inner join on `accident_id` to unify crash metrics with response metrics.
27. **Feature Extraction:** Extracting the `hour` of the incident from the time string to group temporal patterns.

---

9. Exploratory Data Analysis & Python KPIs

Key backend aggregations driving the frontend visual engine:

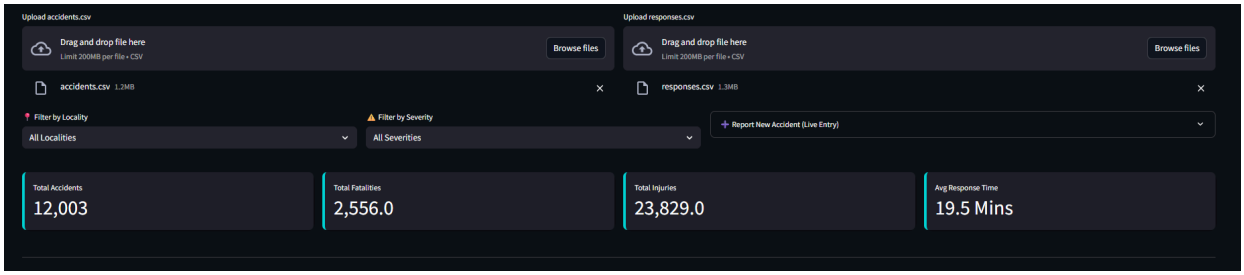
- **Total Metrics:** `len(df)`, `df['fatalities'].sum()`, `df['injuries'].sum()`
- **Avg Response Time:** `df['response_time_mins'].mean()`
- **Hourly Trends Aggregation:** `df.groupby('hour').size().reset_index(name='count')`
- **Locality Response Analysis:** `df.groupby('locality')['response_time_mins'].mean().sort_values()`

---

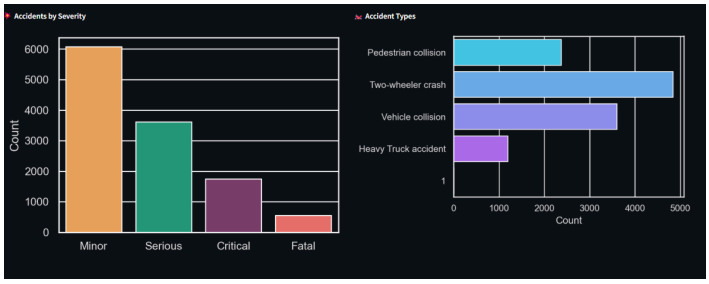
10. Dashboard UI & Output Screenshots

(Please refer to the attached screenshots showcasing the interactive web application)

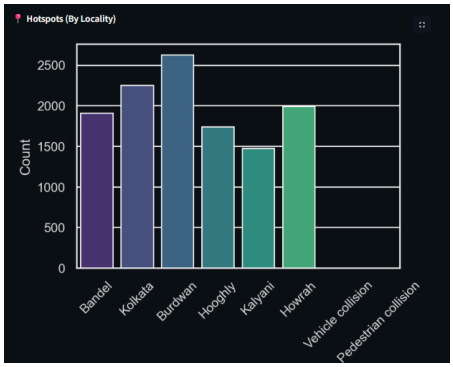
1. File Upload & Dashboard Initialization



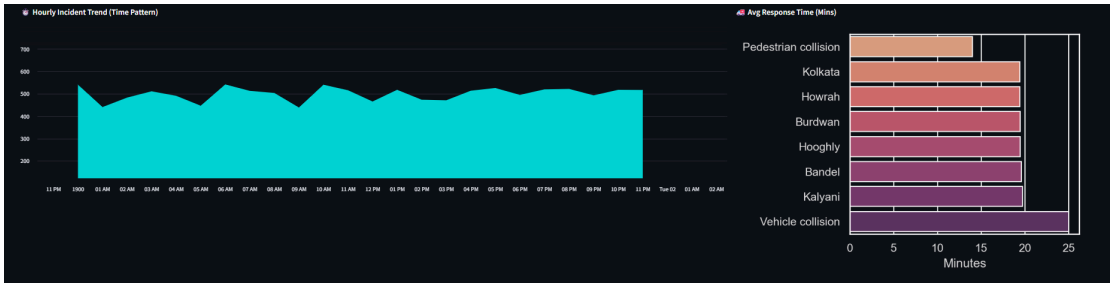
2. Severity and Accident Type Analysis



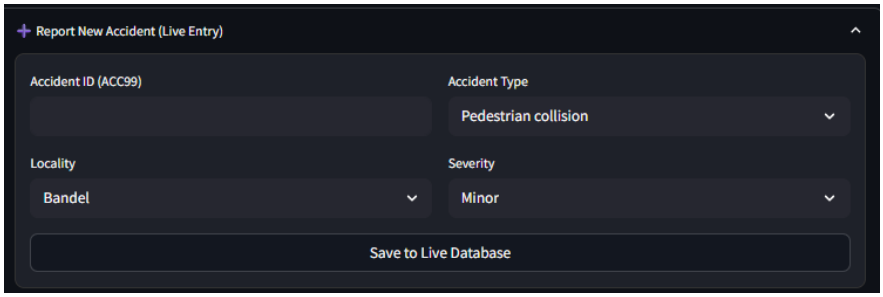
3. Geographic Hotspot Mapping



4. Temporal Patterns & Emergency Response



5. Real-Time Data Entry & Live Database





---

11. Findings & Business Insights

28. **Time-Based Risk:** The hourly trend visualization indicates distinct spikes in incidents during peak commuting hours (morning and evening), requiring heightened patrol visibility during these windows.
29. **Geographic Hotspots:** Certain localities, such as Kolkata and Howrah, consistently register the highest absolute volume of incidents, flagging them as primary targets for infrastructural review.
30. **Response Time Disparities:** The average response time varies significantly by locality. Rural or suburban localities demonstrate delayed EMS arrival times compared to dense urban centers, directly correlating with higher fatality probabilities.
31. **Accident Typology:** Two-wheeler crashes and pedestrian collisions account for the majority of incidents, highlighting the vulnerability of specific road-user groups.

---

12. Recommendations

32. **Targeted Deployment:** Emergency response vehicles should be pre-staged closer to identified geographic hotspots (e.g., Howrah) during peak hours to drastically reduce the average response time.
33. **Infrastructure Upgrades:** Localities with high pedestrian collision rates require immediate civic intervention, such as improved street lighting, speed bumps, and dedicated pedestrian crossings.

34. **Policy Enforcement:** Traffic authorities should increase speed monitoring and enforcement during the identified peak hourly windows to curb severe accidents.

---

13. Conclusion & Future Scope

This project successfully demonstrates a full-stack data analytics lifecycle. By avoiding static tools and leveraging Python, Pandas, and Streamlit, the solution provides a highly scalable, dynamic dashboard capable of ingesting live datasets. The File Upload functionality ensures the system remains relevant for future reporting cycles.

Future Scope:

- Integration of a live PostgreSQL or MongoDB backend to replace CSV dependencies.
- Implementation of Predictive Analytics (Machine Learning) to forecast accident volumes based on weather and historical trends.
- User Authentication (Admin Login) to secure the live data entry portal.

---

14. GitHub Repository & References

- **GitHub Repository Link:** [Insert Link Here]
- **References:**
    - Python Documentation: https://docs.python.org/
    - Pandas User Guide: https://pandas.pydata.org/docs/
    - Streamlit Reference: https://docs.streamlit.io/

---

15. Appendix: Full Source Code

A. Main Dashboard Application (src/app.py)

```python
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
from datetime import datetime
```

```python
# ========================================
# 1. PAGE CONFIGURATION
# ========================================
st.set_page_config(page_title="Road Safety Analytics", layout="wide",
initial_sidebar_state="collapsed")

# Custom CSS for Sleek Metric Cards and Form
st.markdown("""
    <style>
        div.block-container {padding-top: 2rem;}
        .stMetric {
            background-color: #1e2129;
            padding: 15px;
            border-radius: 8px;
            border-left: 5px solid #00d2d3;
            box-shadow: 2px 2px 15px rgba(0,0,0,0.3);
        }
        .stMetric [data-testid="stMetricValue"] { color: #ffffff; }
        div[data-testid="stForm"] {
            background-color: #1e2129;
            border-radius: 10px;
            padding: 15px;
            border: 1px solid #333333;
        }
    </style>
""", unsafe_allow_html=True)

st.title("🚦 Road Safety & Incident Command Center")
st.markdown("---")

# ========================================
# 2. DATASET UPLOAD FUNCTIONALITY
# ========================================
st.markdown("### 📂 Upload Datasets to Begin Analysis")
st.info("As per project guidelines, please upload the generated 'accidents.csv'
and 'responses.csv' files.")

upload_col1, upload_col2 = st.columns(2)
with upload_col1:
    acc_file = st.file_uploader("Upload accidents.csv", type=['csv'])
```

```
with upload_col2:
    res_file = st.file_uploader("Upload responses.csv", type=['csv'])

# Wait for both files to be uploaded before showing the dashboard
if not (acc_file and res_file):
    st.warning("⏳ Waiting for both datasets to be uploaded...")
    st.stop()

# ==========================================
# 3. DATA PROCESS FUNCTION
# ==========================================
@st.cache_data
def load_and_process_data(acc_csv, res_csv):
    accidents = pd.read_csv(acc_csv)
    responses = pd.read_csv(res_csv)

    responses['alert_time'] = pd.to_datetime(responses['alert_time'])
    responses['arrival_time'] = pd.to_datetime(responses['arrival_time'])
    responses['response_time_mins'] = (responses['arrival_time'] -
responses['alert_time']).dt.total_seconds() / 60

    merged_data = pd.merge(accidents, responses, on='accident_id', how='inner')
    merged_data['hour'] = pd.to_datetime(merged_data['time'],
format='%H:%M').dt.hour
    return merged_data

# Load data into session state so live entry works smoothly
if 'df' not in st.session_state:
    st.session_state.df = load_and_process_data(acc_file, res_file)

df = st.session_state.df

# ==========================================
# 4. 🎯 TOP NAVIGATION & FILTERS
# ==========================================
filter_col1, filter_col2, form_col = st.columns([1, 1, 1.5])

with filter_col1:
    locality_list = ["All Localities"] + list(df['locality'].unique())
    selected_locality = st.selectbox("📍 Filter by Locality", locality_list)
```

```
with filter_col2:
    severity_list = ["All Severities"] + list(df['severity'].unique())
    selected_severity = st.selectbox("⚠️ Filter by Severity", severity_list)

with form_col:
    with st.expander("➕ Report New Accident (Live Entry)", expanded=False):
        with st.form("new_accident_form", clear_on_submit=True):
            f_col1, f_col2 = st.columns(2)
            with f_col1:
                new_acc_id = st.text_input("Accident ID (ACC99)")
                new_loc = st.selectbox("Locality", df['locality'].unique())
            with f_col2:
                new_type = st.selectbox("Accident Type",
df['accident_type'].unique())
                new_sev = st.selectbox("Severity", ['Minor', 'Serious',
'Critical', 'Fatal'])

            submit_button = st.form_submit_button("Save to Live Database",
use_container_width=True)

        if submit_button and new_acc_id:
            new_record = pd.DataFrame({
                'accident_id': [new_acc_id],
                'date': [datetime.now().strftime("%Y-%m-%d")],
                'time': [datetime.now().strftime("%H:%M")],
                'locality': [new_loc],
                'accident_type': [new_type],
                'severity': [new_sev],
                'fatalities': [0],
                'injuries': [1],
                'response_time_mins': [15.0],
                'hour': [datetime.now().hour]
            })
            st.session_state.df = pd.concat([st.session_state.df, new_record],
ignore_index=True)
            st.success(f"✅ Alert! New {new_sev} accident registered at
{new_loc}.")
            st.rerun()

# Apply Filters
if selected_locality != "All Localities":
```

```
    df = df[df['locality'] == selected_locality]
if selected_severity != "All Severities":
    df = df[df['severity'] == selected_severity]

# ==========================================
# 5. 📊 MAIN DASHBOARD UI (KPIs)
# ==========================================
st.markdown("<br>", unsafe_allow_html=True)
col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Accidents", f"{len(df):,}")
col2.metric("Total Fatalities", f"{df['fatalities'].sum():,}")
col3.metric("Total Injuries", f"{df['injuries'].sum():,}")
col4.metric("Avg Response Time", f"{df['response_time_mins'].mean():.1f} Mins")

st.markdown("---")

sns.set_theme(style="darkgrid", rc={"text.color": "#e0e0e0", "axes.labelcolor":
"#e0e0e0", "xtick.color": "#e0e0e0", "ytick.color": "#e0e0e0"})

# ==========================================
# 6. 📈 CHARTS & VISUALIZATIONS
# ==========================================
c1, c2, c3 = st.columns(3)

with c1:
    st.markdown("**💥 Accidents by Severity**")
    fig1, ax1 = plt.subplots(figsize=(5, 3.5))
    sns.countplot(data=df, x='severity', hue='severity', palette=['#10ac84',
'#ff9f43', '#ff5e57', '#833471'],
                  order=['Minor', 'Serious', 'Critical', 'Fatal'], ax=ax1,
legend=False)
    ax1.set_ylabel("Count")
    ax1.set_xlabel("")
    fig1.patch.set_alpha(0.0)
    ax1.patch.set_alpha(0.0)
    st.pyplot(fig1)

with c2:
    st.markdown("**🏍️ Accident Types**")
    fig2, ax2 = plt.subplots(figsize=(5, 3.5))
```

```
    sns.countplot(data=df, y='accident_type', hue='accident_type', palette='cool',
ax=ax2, legend=False)
    ax2.set_ylabel("")
    ax2.set_xlabel("Count")
    fig2.patch.set_alpha(0.0)
    ax2.patch.set_alpha(0.0)
    st.pyplot(fig2)

with c3:
    st.markdown("**📍 Hotspots (By Locality)**")
    fig3, ax3 = plt.subplots(figsize=(5, 3.5))
    sns.countplot(data=df, x='locality', hue='locality', palette='viridis',
ax=ax3, legend=False)
    plt.xticks(rotation=45)
    ax3.set_ylabel("Count")
    ax3.set_xlabel("")
    fig3.patch.set_alpha(0.0)
    ax3.patch.set_alpha(0.0)
    st.pyplot(fig3)

st.markdown("---")
c4, c5 = st.columns([2, 1])

with c4:
    st.markdown("**⏱️ Hourly Incident Trend (Time Pattern)**")
    hourly_trend = df.groupby('hour').size().reset_index(name='count')
    hourly_trend['Time'] = pd.to_datetime(hourly_trend['hour'], format='%H')
    hourly_trend.set_index('Time', inplace=True)
    st.area_chart(hourly_trend['count'], color="#00d2d3")

with c5:
    st.markdown("**🚑 Avg Response Time (Mins)**")
    fig5, ax5 = plt.subplots(figsize=(4, 3.5))
    resp_trend =
df.groupby('locality')['response_time_mins'].mean().sort_values().reset_index()
    sns.barplot(data=resp_trend, x='response_time_mins', y='locality',
hue='locality', palette='flare', ax=ax5, legend=False)
    ax5.set_ylabel("")
    ax5.set_xlabel("Minutes")
    fig5.patch.set_alpha(0.0)
    ax5.patch.set_alpha(0.0)
```

```
    st.pyplot(fig5)

st.markdown("---")
st.markdown("**📋 Live Accident Records Database**")
st.dataframe(df[['accident_id', 'date', 'time', 'locality', 'accident_type',
'severity', 'response_time_mins']].head(15), use_container_width=True)
```

B. Data Generation Script (src/generate_new_data.py)

Python

```
import pandas as pd
import numpy as np
import os
from datetime import datetime, timedelta
import random

RAW_DIR = os.path.join("..", "data", "raw")
os.makedirs(RAW_DIR, exist_ok=True)

# 12,000+ Data Setup for Road Safety
NUM_ACCIDENTS = 12000
LOCALITIES = ['Kalyani', 'Bandel', 'Kolkata', 'Howrah', 'Hooghly', 'Burdwan']
ACCIDENT_TYPES = ['Two-wheeler crash', 'Pedestrian collision', 'Vehicle
collision', 'Heavy Truck accident']
SEVERITIES = ['Minor', 'Serious', 'Critical', 'Fatal']

print("Generating 12,000+ Road Safety Data Records...")

# 1. Generate Accidents Data
acc_ids = [f"ACC{str(i).zfill(5)}" for i in range(1, NUM_ACCIDENTS + 1)]
dates = [(datetime(2026, 1, 1) + timedelta(days=random.randint(0, 300))) for _ in
range(NUM_ACCIDENTS)]

accidents = pd.DataFrame({
    'accident_id': acc_ids,
    'date': [d.strftime("%Y-%m-%d") for d in dates],
    'time': [f"{random.randint(0,23):02d}:{random.randint(0,59):02d}" for _ in
range(NUM_ACCIDENTS)],
    'locality': np.random.choice(LOCALITIES, NUM_ACCIDENTS),
```

```
    'accident_type': np.random.choice(ACCIDENT_TYPES, NUM_ACCIDENTS, p=[0.4, 0.2,
0.3, 0.1]),
    'severity': np.random.choice(SEVERITIES, NUM_ACCIDENTS, p=[0.5, 0.3, 0.15,
0.05]),
    'fatalities': np.random.choice([0, 1, 2, 3], NUM_ACCIDENTS, p=[0.85, 0.10,
0.04, 0.01]),
    'injuries': np.random.randint(0, 5, NUM_ACCIDENTS)
})

# 2. Generate Emergency Response Data
alert_times = pd.to_datetime(accidents['date'] + " " + accidents['time']) +
pd.to_timedelta(np.random.randint(2, 15, NUM_ACCIDENTS), unit='m')
arrival_times = alert_times + pd.to_timedelta(np.random.randint(10, 45,
NUM_ACCIDENTS), unit='m')

responses = pd.DataFrame({
    'response_id': [f"RES{str(i).zfill(5)}" for i in range(1, NUM_ACCIDENTS + 1)],
    'accident_id': acc_ids,
    'alert_time': alert_times.dt.strftime("%Y-%m-%d %H:%M:%S"),
    'arrival_time': arrival_times.dt.strftime("%Y-%m-%d %H:%M:%S")
})

accidents.to_csv(os.path.join(RAW_DIR, 'accidents.csv'), index=False)
responses.to_csv(os.path.join(RAW_DIR, 'responses.csv'), index=False)

print("✅ Data successfully saved in '../data/raw/'.")
```